

Mini projet B :

L'objectif de ce projet est de comparer différentes méthodes de calcul d'intégrales en python en fonction de leur rapidité et de leur précision.

1 Structure du programme :

Le programme est composé d'un main.py comprenant l'interface utilisateur et de 6 modules :

- Un module moindre_rectangle.py pour calculer les intégrales par la méthode des moindres rectangles avec python natif et numpy
- Un module trapeze_python.py pour calculer les intégrales par la méthode des trapèzes avec python natif
- Un module trapeze_numpy.py pour calculer les intégrales par la méthode des trapèzes avec numpy
- Un module simpson.py pour calculer les intégrales par la méthode de simpson avec python natif et numpy
- Un module performance et un module graphique.py pour calculer les performances et dessiner les graphiques

2 Méthodologie

2.1 Méthode analytique

La méthode de calcul analytique est définie dans le module performance.py. Son but est de calculer une intégrale de manière analytique, c'est-à-dire en prenant la primitive du polynôme et en faisant la différence entre la primitive à la borne supérieure et à la borne inférieure. Ce résultat sera considéré comme la référence et le calcul de l'erreur sera fait par rapport à cette valeur analytique.

2.2 Moindre rectangle

Pour les moindres rectangles, on définit la largeur des rectangles par l'intervalle divisé par le nombre de segments, tandis que la hauteur est calculée en prenant la valeur moyenne entre le début du segment et la fin.

2.3 Trapèze

La méthode des trapèzes, la largeur des segments est calculée de la même manière avec l'intervalle. Pour la hauteur, on calcule la hauteur à gauche et à droite du segment puis on utilise la formule de calcul de l'aire donnée dans la définition du projet.

2.4 Simpson

La méthode de Simpson utilise une formule approchée tel que :

$$\int_a^b f(x)dx \approx \frac{b-a}{6} + (f(a) + 4f\left(\frac{a+b}{2}\right) + f(b))$$

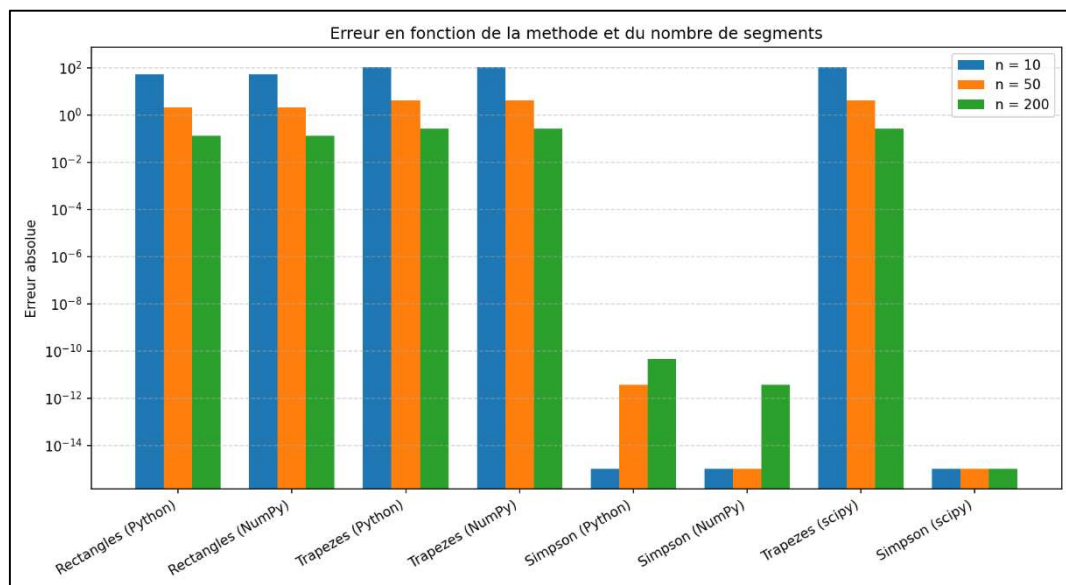
3 Analyse des résultats

Pour l'exemple, on prend un polynôme tel que $f(x) = 1 + 2x + 3x^2 + 4x^3$ et on intègre de 0 à 10.

3.1 Comparaison de l'erreur

Le graphique de l'erreur en fonction de la méthode démontre clairement que les méthodes des moindres rectangles et des trapèzes sont moins précises que la méthode de Simpson. Pour Simpson l'erreur absolue est quasiment nulle peu importe le nombre de n (échelle logarithmique) tandis que pour les trapezes elle est de l'ordre de la centaine pour 10 segments. Il est intéressant de remarquer que l'utilisation de python ou numpy sur ces 3 méthodes a très peu d'impact sur les résultats.

Entre la méthode des trapèzes et des rectangles, il faut aussi voir que la méthode des rectangles est très légèrement plus précise.

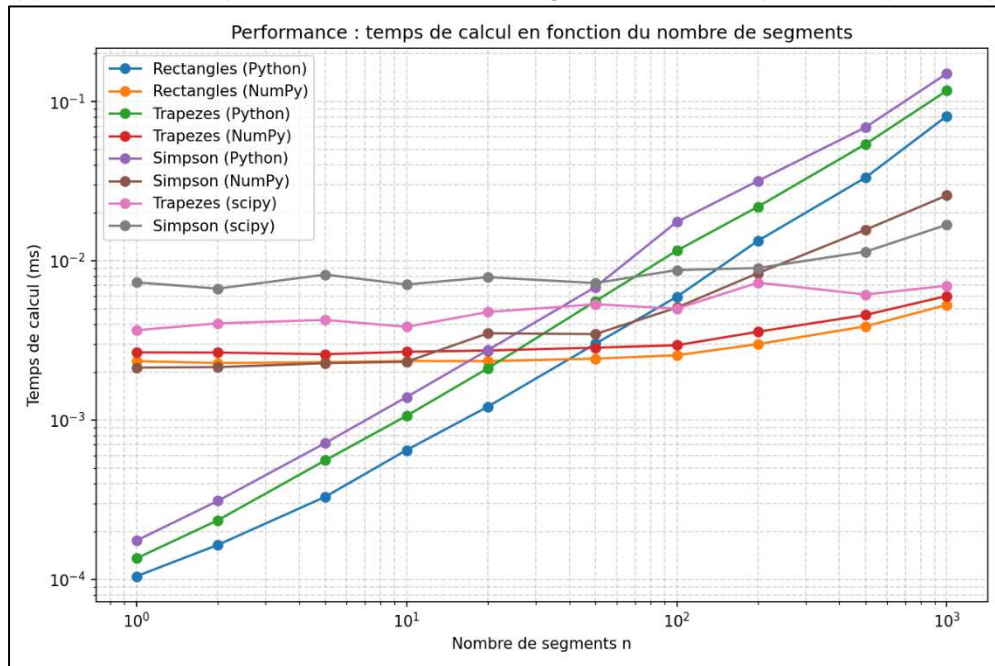


Enfin, ce graphique nous permet aussi de voir l'utilisation scipy est très précise pour la méthode Simpson mais que cela a des résultats similaires pour la méthode trapèze avec l'utilisation de python et numpy.

3.2 Comparaison du temps de calcul

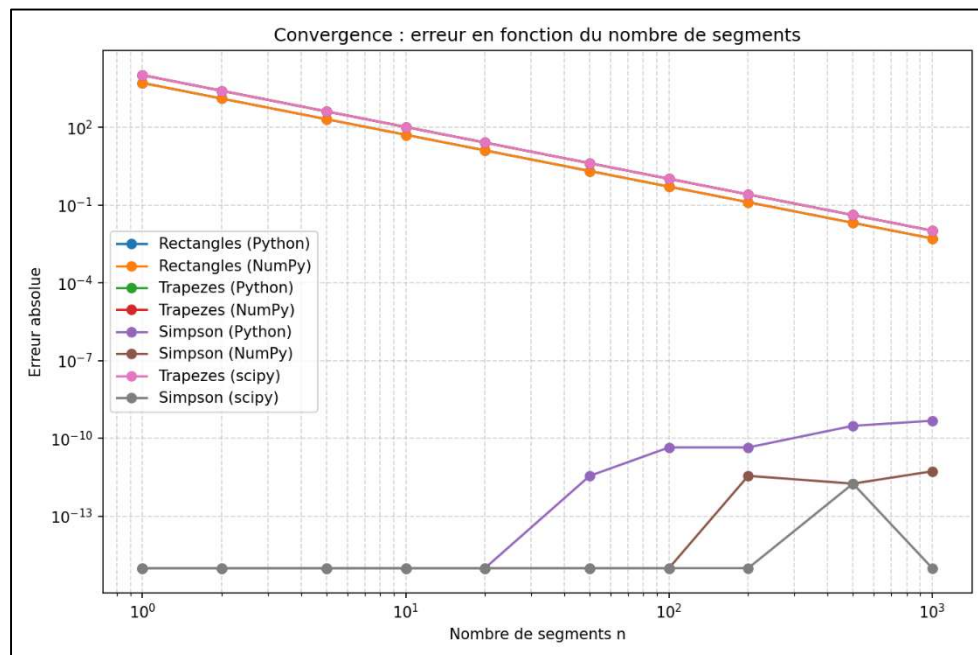
La courbe du temps de calcul est très intéressante à analyser. La première chose à remarquer est que le temps d'exécution d'une méthode avec numpy est indépendant du nombre de segments, alors que le temps d'exécution d'une méthode avec Python est proportionnel au nombre de segments. Cela signifie que pour une approximation rapide avec un faible nombre de segments, Python est plus rapide, mais si l'on désire une valeur précise (c.-à-d. un grand nombre de segments), numpy est plus efficace.

Ensuite, assez logiquement, le temps d'exécution de la méthode de Simpson est supérieur à celui de la méthode des trapèzes, qui lui-même est supérieur à la méthode des rectangles. Enfin, `scipy` est aussi indépendant du nombre de segments mais est plus lent que `numpy`.



3.3 Convergence

Pour la convergence, le graphique doit être interprété prudemment, en réalité les courbes en dessous des puissances -7 sont en fait des erreurs quasiment nulles mais l'échelle logarithmique nous trompe facilement. Autrement dit, la méthode de Simpson a des erreurs tellement faibles que ces valeurs convergent très rapidement.



Legavre Kilian
Minashvili Nino

Elmirzoiev Pradel Victor

Pour ce qui est des autres courbes, les courbes de trapèzes avec python, numpy et scipy sont toutes confondues (courbe rose), confirmant que peu importe la manière de le coder, cette méthode a des résultats similaires.

La méthode des rectangles a un comportement similaire à celle des trapèzes, les courbes de convergence avec python et numpy sont confondues dans la courbe orange, ces courbes confirment que la méthode des rectangles est un peu plus précise que celles de trapèzes.

Pour la méthode des trapèzes comme des rectangles, un niveau de convergence suffisant est obtenu avec $n > 50$.

4 Conclusion

Ce projet nous a permis de bien comprendre l'intérêt de la programmation python pour le calcul, et de comprendre que ce langage peut totalement avoir les mêmes fonctions que Matlab sans la lourdeur du logiciel. Ce projet nous a permis de bien comprendre l'intérêt de la programmation Python pour le calcul, et de comprendre que ce langage peut totalement avoir les mêmes fonctions que MATLAB sans la lourdeur du logiciel.

L'analyse des résultats démontre clairement que la méthode de Simpson surpasse toutes les autres méthodes en termes de précision pour un temps d'exécution très légèrement supérieur.